

Hermes — Briefing Sessão 23/04/2026

Continuação da implementação do patch `run_agent.py` — injeção semântica automática

Estado do sistema (fim de 22/04)

Componente	Status	Detalhe
vault-first	OK	Funcionando, SOUL.md atualizado
reindex_vault	OK	172/173 notas indexadas — MAX_CHARS=3500
embeddings semanticos	OK	vec_obsidian populada, limite API corrigido
patch run_agent.py	PENDENTE	Tarefa desta sessao
populate_sessions	OK parcial	1 sessao migrada (dry-run)
compactacao	N/A	Parametro e ratio, nao tokens

Tarefa desta sessao: patch run_agent.py

Objetivo: injetar automaticamente contexto semantico do vault antes de cada chamada ao modelo Kimi. Quando pronto, o Hermes chegara em cada conversa ja sabendo o que estava sendo feito na sessao anterior.

O que ja esta pronto

- `get_semantic_context()` criada e testada com sucesso
- `HERMES_SEMANTIC_CONTEXT=true` adicionado ao `~/hermes/.env`
- Ponto de injecao identificado: apos `memory_manager.prefetch_all()`
- 172 notas indexadas e prontas para busca semantica

O que falhou na ultima tentativa

O patch foi aplicado via script de injecao por linha (`/tmp/inject_semantic.py`) mas gerou problema de indentacao mista (tabs + espacos). Foi revertido via git checkout.

Abordagem correta para esta sessao

Usar `str_replace` cirurgico — nao injecao por linha. Fazer git commit de backup ANTES de qualquer mudanca.

Instrucoes para o Hermes

Passo 1 — Backup obrigatorio

```
cd ~/.hermes/hermes-agent && git add -A && git commit -m "pre-semantic-inject backup $(date +%Y%m%d_%H%M)"
```

Passo 2 — Localizar ponto de injecao

```
grep -n "prefetch_all\|original_user_message\|ext_prefetch"
~/hermes/hermes-agent/run_agent.py | head -20
```

Procurar o bloco exato que contem `memory_manager.prefetch_all()` e o bloco ephemeral onde `ext_prefetch_cache` e injetado. Anotar os numeros de linha.

Passo 3 — Mostrar `old_str` ANTES de aplicar

Antes de qualquer `str_replace`: mostrar o trecho exato que sera substituido (`old_str` completo) e o novo trecho (`new_str`). Aguardar confirmacao logica antes de aplicar.

Passo 4 — O `new_str` deve ter este formato

```
# --- injecao semantica --- if os.getenv('HERMES_SEMANTIC_CONTEXT') == 'true': try:
sys.path.insert(0, os.path.expanduser('~/.hermes/skills/obsidian/scripts')) from
search_semantic import get_semantic_context sem_ctx =
get_semantic_context(original_user_message, limit=3, max_chars=2000) if sem_ctx:
notas = '\n\n'.join(f"### {n['title']}\n\n{n['content']}" for n in sem_ctx)
ext_prefetch_cache += f'\n\n## Notas relevantes do vault\n{n[notas]}' except
Exception: pass # --- fim injecao semantica ---
```

Passo 5 — Verificar sintaxe apos patch

```
python3 -m py_compile ~/.hermes/hermes-agent/run_agent.py && echo "OK"
```

Passo 6 — Rollback se qualquer erro

```
cd ~/.hermes/hermes-agent && git checkout run_agent.py
```

Se houver erro de indentacao, `IndentationError`, ou `SyntaxError`: executar rollback imediatamente. NAO tentar corrigir manualmente. Registrar o erro e parar.

Formato do relatorio final

Campo	O que reportar
Commit de backup	Hash do commit pre-patch
<code>old_str</code> aplicado	Primeiras e ultimas 2 linhas do trecho substituido
<code>new_str</code> aplicado	Primeiras e ultimas 2 linhas do trecho inserido
Linhas alteradas	Output do git diff --stat apos patch
Sintaxe	python3 -m py_compile — OK ou erro exato
Teste funcional	Resultado de <code>get_semantic_context("embeddings")</code> apos patch
Rollback?	Sim/Nao — se sim, motivo e estado final

Contexto tecnico relevante

Arquivos importantes

- `run_agent.py` — `~/.hermes/hermes-agent/run_agent.py` (~10k linhas)
- `search_semantic.py` — `~/.hermes/skills/obsidian/scripts/search_semantic.py`
- `.env` — `~/.hermes/.env` (contem `HERMES_SEMANTIC_CONTEXT=true`)
- `memory_store.db` — `~/.hermes/memory/memory_store.db` (vec_obsidian com 172 registros)

Assinatura da funcao `get_semantic_context`

```
get_semantic_context(query: str, folder: str = None, limit: int = 5, max_chars: int
= 3000) -> list[dict]
```

Retorna lista de dicts com: filepath, title, folder, tags, score, content

Variavel `original_user_message`

Contem a mensagem original do usuario antes de qualquer processamento. E a variavel correta para passar para `get_semantic_context` como `query`.

Lembrete: o patch do `run_agent.py` e a ultima pendencia critica. Quando entrar, o Hermes vai chegar em cada conversa ja com contexto semantico pre-carregado. Sistema estimado em 8.0/10 — este patch leva para 9.0.